

ABYSS

게임 콘텐츠를 구현하고,
변경에 강한 시스템을 설계합니다.

Darkest Dungeon 재구현 · Unity / C# 포트폴리오

UNITY 6

C#

PURE CORE

SAVE v27

GAME PROGRAMMING



ABYSS / 전투 화면

CODE / SYSTEMS / RELIABILITY

GAME PROGRAMMER / PROJECT PORTFOLIO

ABYSS 프로젝트 소개

Darkest Dungeon의 전투·탐험·영지 운영을 Unity와 C#으로 재구현한 학습·포트폴리오 프로젝트입니다.
원작과 기존 Unity 재현 프로젝트를 분석한 역기획 문서를 바탕으로 구현했습니다.



원정 지도 / 던전 선택과 1~4인 파티 편성

원작: Darkest Dungeon (다키스트 던전) / Red Hook Studios

프로젝트 구현 범위

C# 전투·탐험·캠페인 시스템, 저장·복원, Unity UI 연결, 개발 도구 및 검증 자동화

개발 기간: 2026.07.31-현재 · 개인 프로젝트(1인)

사용 기술

C# · Unity 6 · URP · uGUI · Input System
JSON · .NET · NUnit · Git · PowerShell · Codex

게임 흐름

편성·보급 → 탐험·전투 → 성공 귀환 / 후퇴
→ 결산·성장·영지 운영

콘텐츠·시스템 구현

전투 효과 실행기와 설정 기반 코어

코드 분석·문제 해결

의존성 분리, 난수 격리, 저장 실패 복구

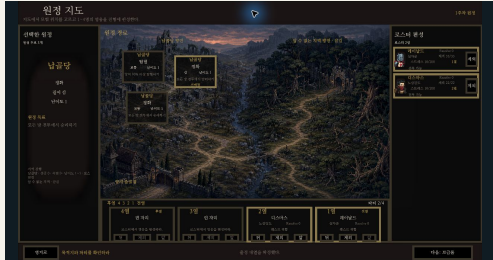
도구·자동화

썬 생성, 데이터 검증, AI 결과 검증

플레이 흐름과 완료 범위

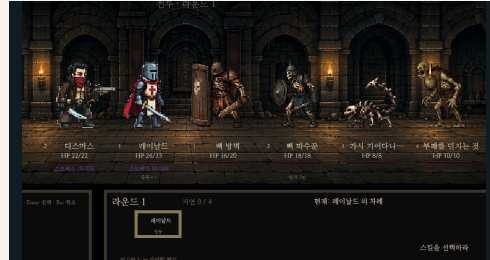
같은 캠페인에서 편성·전투·후퇴 귀환을 진행하고, 저장 후 재실행해 영웅 상태와 자원 복원을 확인했습니다.

01 편성·출발



레이날드·디스마스 2인 편성 → 납골당 출정
보급비 485 지출 / 금화 1,000 → 515

02 전투 입력



디스마스의 사악한 베기 → 뼈 방벽 4 피해
적 HP 20 → 16 / 전투 로그에 결과 기록

03 후퇴 귀환·결산



전투·원정 후퇴 → 전투 1회·방 2곳 결산
금화 보상 0 / 디스마스 기벽 「허약」 획득

04 저장 후 재실행



영지 저장 → Play Mode 종료·재실행
슬롯 2 재개 → 2주차 영지 상태 복원

저장 직전과 재실행 후 / 레이날드 상태 확대

영지에서 저장 직전



슬롯 2 재개 후



복원 후 다시 저장한 파일도 원본과 바이트 단위로 일치했습니다.

확인 범위: 2인 편성·보급 → 탐험·전투 → 후퇴 귀환 → 저장·재개 | 확인 방법: 실제 UI 입력 및 저장 전후 비교

검증 환경: Windows · Unity 6000.3.13f1 Editor Play Mode / 확인일: 2026.09.07 / 확인 코드: ca3674f

SOURCE / 동일 캠페인 단계별 원본 캡처와 저장 상태 비교

저장 전후 상태 대조 / 전 항목 일치

항목	저장 직전	재실행 후
영웅	레이날드·디스마스	레이날드·디스마스
HP	33/33 · 22/22	33/33 · 22/22
스트레스	25 · 25	25 · 25
금화	515	515
주차·위치	2주차 · 영지	2주차 · 영지

규칙과 화면의 책임을 분리한 구조

전투·탐험·저장을 담당하는 C# 코드를 Core로 모으고, Unity 화면과 독립적으로 실행·검증합니다.

Unity Presentation

화면·입력·씬 전환



Application

캠페인 세션 / 명령 처리 / 읽기 전용 화면 데이터



Content

JSON 로드·검증 / 게임별 설정 / 전투 데이터 조립



Infrastructure

저장 데이터 변환 / 파일 입출력 / JSON 지원



Rules

전투·탐험 계산 / 게임 상태 모델 / 결과 이벤트

왜 분리했는가

씬·프레임·UI 이벤트와 전투 계산이 섞이면 재현과 회귀 확인이 어려워집니다. 같은 규칙을 콘솔과 Unity에서 실행하도록 경계를 만들었습니다.

경계를 강제하는 장치

계층별 허용 참조를 빌드 설정과 대조하는 자동 검사(A15)를 둡니다. 규칙 코드의 Unity·파일 입출력·시간·직접 난수 생성도 제한합니다.

직접 참조 집합

Rules	없음
Infrastructure	Rules
Content	Rules, Infrastructure
Application	Rules, Infrastructure, Content
Presentation	Rules, Content, Application

화살표: 핵심 계층 방향. 정확한 직접 참조는 우측 표 기준.

설계 결과 / 화면 없이 전투 계산을 검증하고, UI에는 읽기 전용 조회 데이터(DTO)와 명령을 제공합니다.

SOURCE / Core/Rules/Abys.Core.Rules.asmdef · verify.ps1

11종 스킬 효과를 하나의 실행 경로로

효과 객체와 실행 단계를 분리해 버프·이동·치료·소환을 조합할 수 있도록 구현했습니다.



DamageOverTimeSkillEffect.Apply / 실제 코드

```

context.Battle.StatusesOf(context.Target.Id)
    .GetOverTime(Status)
    .AddStack(DamagePerTurn, Turns);
context.Log.Publish(new StatusAppliedEvent(
    context.Target.Id,
    context.Naming.GetName(context.Target),
    Status, DamagePerTurn, Turns));
  
```

명중 후 지속 피해 스택과 종류·대상·수치가 정해진 이벤트를 기록합니다. 실행기가 단계와 순서를 제어하고, 각 효과는 자신의 상태 변경을 수행합니다.

확장 방식

Buff, Push/Pull, Cure, Summon을 기존 기본 판정과 같은 경로에 연결하고, C#의 옛 효과 생성자·변환 표면을 제거했습니다.

실행 순서의 일관성

BattleEngine이 단계와 목록 순서를 관리하고, 실제 효과는 effect.Apply(context)에 위임합니다. 효과를 조합해도 사건과 난수 소비 순서를 유지합니다.

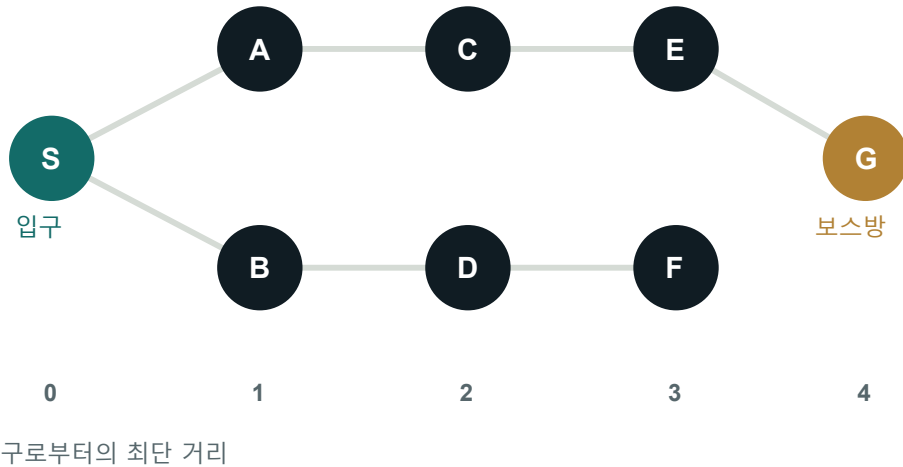
검증

빛나감·반격 사망의 사건 순서, 이동 불가 시 난수 0회, 다중 대상에서도 시전자 이동·소환 1회 실행을 검사합니다.

그래프 탐색으로 던전의 연결성을 보장

방과 복도를 그래프로 모델링하고, BFS로 도달 가능성과 입구에서의 최단 거리를 계산했습니다.

던전 그래프 / 탐색 구조 예시



연결되지 않은 방 보정

Flood 탐색으로 허브에서 도달 가능한 방을 찾고, 인접 칸 연결과 고립 방 정리로 모든 방의 연결성을 보장합니다.

BFS 기반 보스방 배치

Queue 기반 탐색으로 최단 거리를 구한 뒤 입구에서 가장 먼 방을 보스방으로 선택합니다. 동물은 안정 ID 순서로 결정합니다.

자료구조 선택

Queue와 거리 Dictionary로 중복 방문을 막습니다. BFS의 시간 복잡도는 인접 관계 탐색 기준 $O(V + E)$ 입니다.

알고리즘 검증

40개 시드에서 모든 방 도달성 검사 / 20개 시드에서 보스방 최장 거리 검사
같은 시드의 생성 결과를 반복 비교해 지도 생성의 재현성을 확인했습니다.

재현 가능한 전투와 난수 격리

버그를 같은 입력으로 다시 실행하고, 기능 변경이 다른 시스템에 미치는 영향을 통제합니다.

문제

UI 조회나 다른 시스템의 난수 호출이 전투 결과를 바꾸면, 버그를 재현하거나 저장 전후 결과를 비교하기 어렵습니다.

설계

IRandomSource로 난수 생성기(RNG)를 주입하고 PCG32의 진행 상태를 저장합니다. 전투·탐험·전리품 등 7개 용도로 분리해 서로의 난수 소비가 섞이지 않게 합니다.

campaign

state + increment

town

state + increment

dungeon

state + increment

encounter

state + increment

combat

state + increment

ai

state + increment

loot

state + increment

RandomStreamTests.cs:47-48 / 실제 코드

```
RandomStreams changed = new RandomStreams(42UL);
Draw(changed.Get(RandomStreams.Names.Loot), 500);
```

전리품 난수 500회 추가 소비 후에도
전투·던전 난수 수열이 동일한지 대조합니다.

저장 재개 검증

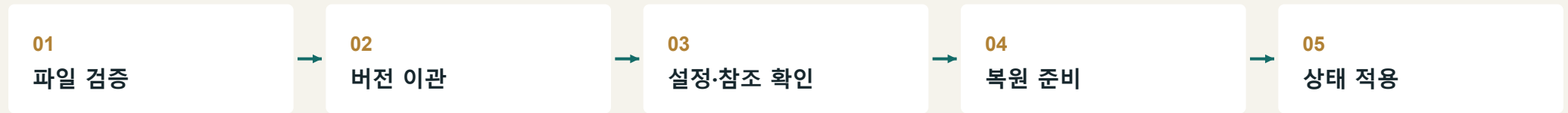
1·2·3·7·11·19턴, 여섯 지점에서 전투를 중단·복원한 결과를 연속 실행과 대조합니다. 모든 스트림 상태의 복원도 검사합니다.

설계 효과와 경계

전리품 변경이 전투 난수에 주는 영향을 격리합니다. 같은 스트림 내부의 소비 순서는 규칙 계약으로 유지합니다.

저장 실패 시 상태 보존과 호환성

저장·복원 과정이 중단되더라도 현재 상태를 보존하도록 검증과 적용 순서를 설계했습니다.



호환성은 버전 번호만으로 판단하지 않습니다

v27 저장에는 콘텐츠 버전과 규칙 설정을 식별하는 값을 담습니다. 전투 수치·진형 크기·원정 설정이 다르면 상태 적용 전에 거절합니다.

과거 규칙의 식별값을 보관합니다

옛 저장을 변환할 때는 그 버전이 사용한 규칙 설정의 식별값을 그대로 보관해 씁니다. 이 값이 동결 서명이며, 저장 당시의 설정을 확인하는 기준입니다.

슬롯 파일 쓰기

임시 파일 작성 → `Flush(true)` →
대상 파일 `Replace / Move` → 백업 정리

Core/Infrastructure/SaveStore.cs:56

복원 실패 시 상태 유지 검증

활동·병동·예약 손상 26종 × 2개 난수 복원 경로, 총 52개 시나리오에서 영지·난수·기존 영웅 참조가 바뀌지 않는지 확인했습니다.

영웅 설정까지 연결

기술 선택·장신구 슬롯·보관함을 보존합니다. 저장 실패 시 영웅 설정 명령의 변경도 되돌립니다.

설계 결과 / 잘못된 저장 데이터는 적용 전에 차단하고, 파일 기록과 메모리 상태의 일관성을 유지합니다.

게임 상태와 사용자 입력을 연결하는 UI

영웅 정보 조회, 기술 선택, 장신구 교환을 Application 서비스와 연결했습니다.



영웅 상태창 / 능력치·기술·장비·기백 정보

씬 제작: 실제 게임의 씬·프리팸을 코드로 생성하는 에디터 도구를 사용해 참조 연결을 유지합니다.

아트 연결: 직업별 고유 ID로 이미지를 선택하고, 누락 시 문자 표시로 조작 기능을 유지합니다.

01 조회가 게임을 바꾸지 않도록

상태창 반복 조회가 캠페인·7개 RNG·저장 파일을 바꾸지 않는지 검사합니다.

02 설정 명령을 실제 저장에 연결

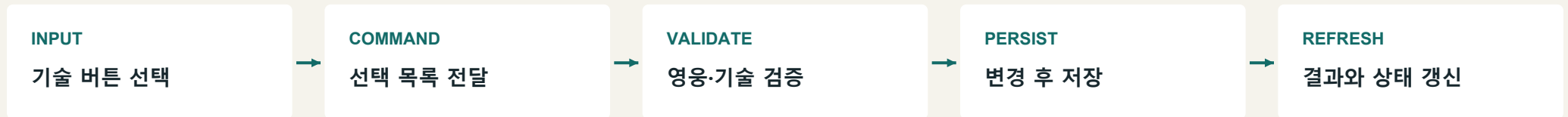
전투 기술 1~4개 제한과 장신구 장착·교체·해제를 서비스로 요청하고 실패 시 복구합니다.

03 입력과 해상도도 계약으로

Esc 닫기 순서, 보관함 항목 제거 후 초점 복구, 1080p/720p 고정 영역·글자 잘림을 확인합니다.

기술 선택부터 저장까지 이어지는 명령 처리

Unity 입력과 게임 상태 변경을 분리하고, 저장 실패 시 이전 설정을 복원합니다.



SaveHeroConfiguration / 오류 결과 반환 줄은 주석으로 축약

```

try
{
    session.SaveCurrent();
    return EstateActionResult.Success(message);
}
catch (Exception exception)
{
    rollback();
    // Failure result returned to UI.
}
  
```

명령 실행 전 이전 값을 보관

기술 선택 순서와 초기화 여부를 캡처합니다. 기술 검증을 통과한 변경만 저장하고, 실패하면 캡처한 값으로 되돌립니다.

화면은 서비스 결과를 다시 조회

UI는 입력을 명령으로 전달합니다. 성공·실패 결과를 표시한 뒤 조회 모델을 다시 받아, 복구된 상태까지 화면에 반영합니다.

실패 경로를 검증한 테스트

저장 오류를 유도한 뒤 기술 선택·장신구 슬롯·보관함 수량·난수 상태를 비교합니다. 입력 처리부터 복구 검사까지 실제 메서드를 제출용 저장소에서 확인할 수 있습니다.

시체 내구도를 생존 HP로 표시하던 오류

사망 표시에 쓰이는 데이터의 의미를 분리하고, 같은 UI 슬롯의 상태 전환을 검증했습니다.



수정 후 화면 / 사망 HP와 시체 내구도를 분리 표시

수정 전 표시

HP 1 / 28



수정 후 표시

HP 0 / 28

시체 내구도 1 / 1

FightUnitView / 표시값 계산식

LivingHealth: IsAlive ? CurrentHealth : 0

CorpseHealth: IsCorpse ? CurrentHealth : 0

원인 / 같은 필드에 다른 의미

시체로 전환된 뒤 CurrentHealth는 파괴용 내구도 1입니다. UI가 이를 생존 최대 체력 28과 함께 읽어 HP 1/28로 표시했습니다.

수정 판단 / 조회 데이터에서 의미 분리

Application 조회에서 생존 HP와 시체 내구도를 나누고, HP 숫자·바·툴팁이 같은 값을 사용하도록 연결했습니다. 파괴용 내구도는 유지합니다.

기대값을 정한 기준

사망 상태의 생존 HP는 0, 시체 내구도는 별도 값입니다. HP 0이어도 살아 있는 영웅이 있으므로 생사 상태를 기준으로 구분합니다.

테스트로 확인한 범위

1칸·2칸 적의 생존 → 시체 → 생존 슬롯 재사용

HP 문자·체력 바·툴팁·시체 그림의 표시 상태

720p·1080p 화면 렌더와 시체 그림의 픽셀 차이

반복 작업을 줄이는 개발 도구

콘텐츠 편집·씬 구성·코드 검증을 도구로 묶어, 변경 결과를 반복해서 확인할 수 있게 만들었습니다.

01 데이터 검증기

JSON 허용 필드·타입·ID 참조를 검사하고, 오류 위치를 파일과 전체 경로로 알려줍니다.

02 씬·프리팹 생성기

에디터 메뉴에서 실제 게임 씬 4개를 생성하고, 자산 식별자(GUID)와 참조를 유지합니다.

03 통합 검증 스크립트

PowerShell로 빌드·규칙 테스트·플레이 흐름·어셈블리 참조 검사를 한 번에 실행합니다.

콘텐츠 수정과 로직 수정을 분리

영웅·스킬·몬스터·시설·이벤트를 JSON으로 정의했습니다. 지원 스키마 안의 콘텐츠 추가와 밸런스 조정은 C# 수정 없이 진행할 수 있습니다.

씬 구성의 재현성 확보

씬 생성 도구가 UI 계층과 오브젝트 참조 연결을 코드대로 만듭니다. Data의 JSON은 빌드 시 Unity의 실행용 데이터 폴더로 복사합니다.

콘텐츠 진단 예시 / 영웅이 존재하지 않는 기술을 참조한 경우

```
heroes.json:$.heroes[0].skills[0]
```

로딩 단계에서 원인과 위치를 식별해, 잘못된 데이터가 전투 실행 오류로 이어지는 것을 차단합니다.

생성형 AI를 활용한 구현과 검증

Codex를 코드 작성·리팩터링·테스트 작성에 활용하고, 규칙 명세와 실행 결과를 기준으로 검증했습니다.

83,449

기대값 비교(assertion) 통과

테스트 안에서 수행한 개별 검사 수

0

계층·완료 조건 위반

의존성 및 플레이 흐름 검사

0 / 0

C# Core 빌드 경고 / 오류

2026.09.07 검증 결과

01 기대 결과 명세화 사망 HP는 0, 시체 내구도는 별도 표시라는 상태별 기준 명시

02 AI 보조 구현 조회 데이터·UI 바인딩·회귀 테스트의 코드 작성에 활용

03 변경 결과 검증 표시 문자열·체력 바·툴팁·화면 픽셀을 각각 검사

리팩터링 검증: 변경 전에 보관한 실행 결과를 기준으로 원정·6주·12주 캠페인을 대조했습니다. 같은 규칙 설정에서 사건 순서·난수 상태·최종 저장이 유지되는지 확인했습니다.

주요 코드 및 설계 사례

독립 실행 코드, 테스트와 핵심 구현 발채를 제출용 저장소에 정리했습니다.

GITHUB REPOSITORY

github.com/darkeye3/Abyss-Portfolio

전투 시스템	구체 효과·단계 실행기·통합 테스트	VIEW
자료구조·알고리즘	BFS 거리 계산 · 독립 실행 샘플	VIEW
결정론	7개 난수 스트림 · 원본 구현 발채	VIEW
저장 안정성	사전 검증·원자적 교체 · 코드 발채	VIEW
Unity·Application	입력·명령·저장·실패 복구의 실제 소스	VIEW
버그 해결	시체 HP 표시 · 원인·수정 판단·검증	VIEW
테스트	공개 샘플 실행 방법과 테스트 구성	VIEW

게임 규칙을 구조화하고, 재현 가능한 코드와 도구로 구현합니다.